

Comparaison des solveurs (CBC, GLPK, Gurobi) pour le Knapsack et le TSP

Hamdane Salsabil , Benaissa Roumeissa

1^{er} octobre 2026

Table des matières

Introduction	2
1 Présentation des solveurs	2
2 Applications aux problèmes classiques/Comparaisons	3
2.1 TSP	3
2.2 Problème du Sac à Dos (Knapsack Problem – KP)	4
2.3 Comparaison globale des solveurs	5
2.4 Conclusion	6
3 La présentation du problème : Vehicle Routing Problem (VRP)	7
3.1 Création des fichiers VRP et validité des données	7
3.2 La modelisation	7
3.3 Tableaux de comparaison	9
3.4 Conclusion	10
3.5 Utilisation des stratégies	10
Conclusion	13

Introduction

Ce rapport présente une analyse complète de trois solveurs largement utilisés dans le domaine de l'optimisation combinatoire et de la programmation mathématique, à savoir :

- **CBC**
- **GLPK**
- **Gurobi**

Chaque algorithme est étudié en relation avec deux problèmes classiques :

- Le problème du sac à dos (*Knapsack Problem*)
- Le problème du voyageur de commerce (*Travelling Salesman Problem – TSP*)

Afin d'obtenir une comparaison pertinente entre les solveurs, nous montrons comment il est possible de définir ou de restreindre les algorithmes utilisés via les options internes de chaque solveur. Cela permet de contrôler précisément la méthode de résolution appliquée à chaque problème, garantissant ainsi une analyse équitable et reproductible.

Enfin, une comparaison expérimentale et conceptuelle entre les trois solveurs sera réalisée en termes de :

- Performance globale
- Capacité à trouver la solution optimale
- Temps d'exécution

1 Présentation des solveurs

Définition générale d'un Solver

Un **solveur** est un outil logiciel permettant de résoudre des problèmes d'optimisation ou mathématiques, tels que :

- Les problèmes linéaires (LP – Linear Programming),
- Les problèmes linéaires en nombres entiers (ILP – Integer Linear Programming),
- Les problèmes non linéaires (NLP – Nonlinear Programming),
- Les problèmes combinatoires (comme le TSP ou le problème du sac à dos).

Le rôle du solveur est de trouver la solution optimale selon un objectif (maximisation ou minimisation d'une fonction) tout en respectant les contraintes du problème.

Exemples de solveurs populaires :

- **GLPK** (GNU Linear Programming Kit)
- **CBC** (Coin-or Branch and Cut)
- **Gurobi**
- **CPLEX** (IBM)
- **SCIP** (Solving Constraint Integer Programs)
- **Mosek** (pour optimisation linéaire et convexe)
- **Xpress** (FICO Xpress Optimization Suite)

Dans notre étude ici on a choisis trois solveurs sont : GLPK, CBC, Gurobi.

Solver CBC

CBC (Coin-or Branch and Cut) est un solveur open-source pour les problèmes de programmation linéaire en nombres entiers (ILP). Il utilise principalement la méthode

Branch and Bound combinée à des **cuts** (coupes linéaires) pour éliminer des solutions non entières et améliorer l'efficacité de la recherche.

- **Méthode principale** : Branch and Bound.
- **Techniques complémentaires** : génération de coupes (cuts), heuristiques, pré-traitement des contraintes.
- **Domaines d'application** : problèmes combinatoires comme le sac à dos, le voyageur de commerce (TSP), et les PLNE en général.

Solver GLPK

GLPK (GNU Linear Programming Kit) est un solveur open-source pour les problèmes de programmation linéaire (PL) et linéaire en nombres entiers (PLNE). Il utilise la **méthode du simplexe** pour les PL et la **méthode Branch and Bound** pour les PLNE.

- **Méthode principale** : Simplexe pour PL, Branch and Bound pour PLNE.
- **Techniques complémentaires** : coupes linéaires (cuts), heuristiques simples.
- **Domaines d'application** : problèmes combinatoires et optimisation linéaire, comme le sac à dos ou le TSP.

Solver Gurobi

Gurobi est un solveur commercial très performant pour les PL, PLNE et autres problèmes d'optimisation mixte. Il combine des techniques avancées de **Branch and Bound**, **coupes** et **heuristiques** pour trouver rapidement la solution optimale.

- **Méthode principale** : Branch and Bound amélioré.
- **Techniques complémentaires** : génération de coupes sophistiquées, heuristiques avancées, prétraitement automatique, parallélisation.
- **Domaines d'application** : optimisation combinatoire et continue, problèmes industriels complexes comme le TSP, le sac à dos, la planification et la logistique.

2 Applications aux problèmes classiques/Comparaisons

2.1 TSP

Le problème du voyageur de commerce (*Travelling Salesman Problem – TSP*) consiste à trouver le chemin le plus court passant exactement une fois par chaque ville et revenant au point de départ.

Formulation ILP (Modèle MTZ) : Soient x_{ij} des variables binaires valant 1 si le trajet passe de la ville i à la ville j , et 0 sinon. L'objectif est de minimiser la distance totale :

$$\min \sum_i \sum_j d_{ij} x_{ij}$$

Sous les contraintes :

- Chaque ville est visitée exactement une fois : $\sum_j x_{ij} = 1$ et $\sum_i x_{ij} = 1$.
- Pas de boucle : $x_{ii} = 0$ pour toute ville i .
- Subtour elimination (MTZ) : $u_i - u_j + nx_{ij} \leq n - 1$ pour $i \neq j$, $i, j \neq$ ville de départ.

Cette formulation permet de résoudre le TSP exact via un solveur ILP.

Fichier	Solver	Distance totale	Temps (s)	Chemin
TSP30	CBC	277	5.60	C1 → C3 → C2 → ...
TSP40	CBC	535	7.83	C1 → C40 → C15 → ...
TSP50	CBC	820	215.55	...
TSP30	Gurobi	277	0.16	C1 → C3 → C2 → ...
TSP40	Gurobi	535	0.61	C1 → C40 → C15 → ...
TSP50	Gurobi	-	-	Version gratuite : limite 50 villes
TSP30	GLPK	277	0.57	C1 → C9 → C7 → ...
TSP40	GLPK	-	-	En cours / trop long
TSP50	GLPK	-	-	En cours / trop long

TABLE 1 – Comparaison des solveurs pour TSP (30, 40, 50 villes). Les chemins complets sont abrégés pour la lisibilité.

Remarque : Parmi les solveurs testés, **Gurobi** fournit les meilleures performances pour les petites et moyennes instances, avec un temps d'exécution beaucoup plus faible. CBC reste performant mais plus lent, tandis que GLPK devient très lent pour des instances supérieures à 30 villes. La version gratuite de Gurobi est limitée pour les instances de 50 villes.

2.2 Problème du Sac à Dos (Knapsack Problem – KP)

Le **problème du sac à dos** consiste à sélectionner un sous-ensemble d'objets parmi un ensemble donné, de manière à maximiser la valeur totale tout en respectant une contrainte de capacité. Chaque objet i a un poids w_i et une valeur v_i . La capacité maximale du sac est C .

Formulation ILP : Soient x_i des variables binaires valant 1 si l'objet i est choisi, et 0 sinon.

Fonction objectif :

$$\max \sum_i v_i x_i$$

Sous les contraintes :

— **Capacité :** le poids total des objets choisis ne doit pas dépasser la capacité :

$$\sum_i w_i x_i \leq C$$

— **Binarité :** chaque objet peut être choisi ou non :

$$x_i \in \{0, 1\}, \quad \forall i$$

Cette formulation permet de résoudre le **Knapsack Problem** exact via un solveur ILP (comme CBC, GLPK ou Gurobi) et de trouver la combinaison d'objets qui maximise la valeur totale sans dépasser la capacité du sac.

Fichier	Solver	la valeur total	Temps (s)	les objets sélectionnés
knapsack50	GLPK	490	0.010789155960083008	6 ,7, 15, 21, 30, 33, 37, 38, 39, 42, 47
knapsack50	CBC	490	0.04375410079956055 s	6 ,7, 15, 21, 30, 33, 37, 38, 39, 42, 47
knapsack50	Gurobi	490	0.01407766342163086 s	6 ,7, 15, 21, 30, 33, 37, 38, 39, 42, 47
knapsack100	GLPK	922	0.010969400405883789 s	1, 6, 16, 20, 22, 25, 26, 27, 29, 41, 43, 51, 55, 59, 66, 67, 69, 73, 76, 80, 83, 84, 86, 93, 96
knapsack100	CBC	922	0.04237842559814453 s	1, 6, 16, 20, 22, 25, 26, 27, 29, 41, 43, 51, 55, 59, 66, 67, 69, 73, 76, 80, 83, 84, 86, 93, 96
knapsack100	Gurobi	922	0.013603687286376953 s	1, 6, 16, 20, 22, 25, 26, 27, 29, 41, 43, 51, 55, 59, 66, 67, 69, 73, 76, 80, 83, 84, 86, 93, 96
knapsack1000	GLPK	9261	0.05586838722229004 s	0, 1, 2, 4, 7, 13, 14, 15, 21, 24, 26, 27, 29, 36, 38, 40, 41, 42, 48, 49, 50, 51, 52, 61, 63, 64, 65, 66, 70, 77, 82, 91, 97, 103, ...
knapsack10000	CBC	9261	0.05586838722229004 s	0, 1, 2, 4, 7, 13, 14, 15, 21, 24, 26, 27, 29, 36, 38, 40, 41, 42, 48, 49, 50, 51, 52, 61, 63, 64, 65, 66, 70, 77, 82, 91, 97, 103, ...
knapsack1000	Gurobi	9261	0.05586838722229004 s	0, 1, 2, 4, 7, 13, 14, 15, 21, 24, 26, 27, 29, 36, 38, 40, 41, 42, 48, 49, 50, 51, 52, 61, 63, 64, 65, 66, 70, 77, 82, 91, 97, 103, ...

TABLE 2 – Comparaison des solveurs pour knapsack probleme avec des différentes nombre d objets.

Remarque

D’après les résultats présentés dans le tableau précédent, tous les solveurs (**GLPK**, **CBC** et **Gurobi**) trouvent la valeur optimale pour chaque instance du problème du sac à dos, garantissant ainsi la qualité des solutions.

En termes de **temps de calcul**, **Gurobi** se distingue par sa rapidité, surtout sur les instances petites et moyennes, et reste compétitif même pour les grandes instances. **CBC** est légèrement plus lent, tandis que **GLPK** est le moins performant.

Ainsi, **Gurobi** peut être considéré comme le meilleur solveur pour ce type de problème, car il allie **exactitude**, **rapidité** et **stabilité**.

2.3 Comparaison globale des solveurs

Après analyse des performances des trois solveurs sur deux problèmes classiques (Knapsack et TSP), on remarque que l’efficacité varie selon la nature du problème et la complexité des contraintes :

- **Problème du Knapsack** : contraintes simples. Tous les solveurs peuvent résoudre les instances jusqu’à 50 ou 100 objets rapidement et avec précision. La différence entre solveurs est limitée.
- **Problème du TSP** : contraintes plus complexes (visiter chaque ville une seule fois, élimination des sous-tours), et le nombre de solutions possibles croît très rapidement avec le nombre de villes. Ici, la différence entre solveurs est notable :
 - GLPK : correct pour de petites instances, mais très lent pour un TSP grand.
 - CBC : performance moyenne, adapté pour des instances de taille moyenne.

- Gurobi : très rapide, supporte les plans de coupes et heuristiques avancés, généralement le meilleur pour les TSP de grande taille.

Problème / Taille	GLPK	CBC	Gurobi
Knapsack (50-100 objets)	Bon, stable	Bon, stable	Excellent
TSP (30 villes)	Moyen	Bon	Très rapide
TSP (40-50 villes)	Lent, parfois incomplet	Moyen-lent	Très rapide, souvent optimal

TABLE 3 – Comparaison des solveurs selon le type de problème et la complexité.

Remarque : Le choix du solveur dépend fortement de la complexité du problème et du nombre de variables. Les problèmes de Knapsack peuvent être résolus facilement par tous les solveurs, tandis que les TSP de grande taille nécessitent un solveur puissant comme Gurobi.

2.4 Conclusion

Performance des solveurs selon la nature du problème :

- Pour les problèmes simples comme le *Knapsack*, n'importe quel solveur (GLPK, CBC, Gurobi) peut les résoudre rapidement et avec précision.
- Pour des problèmes plus complexes comme le *TSP*, surtout avec un grand nombre de villes, la différence entre solveurs est significative. Gurobi est souvent le plus performant, car il est plus rapide et efficace.

Contrôle des performances via les paramètres des solveurs :

- Chaque solveur permet de modifier ses paramètres internes.
- Par exemple, on peut choisir la stratégie de recherche, activer les heuristiques, fixer un temps maximum, etc.
- Cela permet d'améliorer les performances selon la taille et la complexité du problème.

Gestion des *subtours* dans le TSP :

- Dans le TSP, chaque ville doit être visitée exactement une fois sans créer de *subtours* (boucles partielles).
- Plusieurs méthodes existent pour gérer ce problème :
 - Modèle MTZ (utilisé dans cette étude)
 - DFJ Model (Dantzig–Fulkerson–Johnson)
 - *Cutting planes* : ajout de contraintes pour supprimer les boucles et réduire l'espace de recherche
 - *Column generation* : génération progressive de variables pour diminuer la complexité
- Le choix de la méthode dépend du nombre de villes et de la complexité des contraintes, car plus le nombre de villes augmente, plus les *subtours* deviennent problématiques.

Conclusion générale :

- Pour les problèmes simples, n'importe quel solveur est suffisant.
- Pour les problèmes grands et complexes (TSP avec plus de 30-40 villes), Gurobi reste le meilleur choix avec un réglage adéquat des paramètres.
- La méthode de suppression des *subtours* est cruciale pour réduire la complexité et accélérer la résolution.

En résumé, la performance d'un solveur dépend non seulement de son type ou de son coût, mais surtout de la nature du problème et de la complexité de ses contraintes. Le contrôle des paramètres du solveur et le choix d'une bonne méthode pour gérer les *subtours* permettent d'obtenir des solutions plus rapides et stables.

En résumé : le succès dépend de la nature du problème, du choix des outils et des techniques appropriées, pas uniquement du type de solveur.

3 La présentation du problème : Vehicle Routing Problem (VRP)

Le Vehicle Routing Problem (VRP), ou problème de tournées de véhicules, est un problème d'optimisation combinatoire qui consiste à planifier les routes d'une flotte de véhicules afin de servir un ensemble de clients en minimisant un coût global souvent la distance totale.

3.1 Création des fichiers VRP et validité des données

Lors de la création des fichiers VRP, l'objectif est de générer des données **valides** qui peuvent être utilisées directement pour résoudre le problème sans rencontrer de cas *infaisables*. Pour cela, plusieurs étapes et techniques sont appliquées :

1. **Génération des coordonnées dans un intervalle limité** : Les coordonnées (x, y) de chaque client sont choisies dans un petit intervalle pour réduire les distances entre les clients et le dépôt, ce qui permet de rendre les trajets plus réalistes.
2. **Attribution de la demande et du temps de service** : Chaque client reçoit une demande et un temps de service aléatoire dans un intervalle prédéfini, garantissant la diversité des données tout en maintenant la faisabilité.
3. **Création de fenêtres temporelles réalistes (Time Windows)** : Les valeurs de `ready_time` et `due_time` sont calculées en fonction de la distance au dépôt et du temps de déplacement estimé, avec l'ajout d'une marge de sécurité (buffer) pour assurer la faisabilité des visites.
4. **Vérification de la capacité des véhicules** : On s'assure que la somme des demandes ne dépasse pas la capacité totale des véhicules, évitant ainsi des scénarios impossibles à réaliser.
5. **Vérification de la cohérence des fenêtres temporelles** : On garantit que `due_time > ready_time` et que les fenêtres ne sont pas trop étroites par rapport au temps de déplacement, pour permettre la visite de tous les clients.
6. **Répétition des tentatives** : Si les conditions précédentes ne sont pas satisfaites, la génération est répétée plusieurs fois (`max_attempts`) jusqu'à obtenir un scénario valide.
7. **Sauvegarde au format JSON** : Une fois les données validées, elles sont sauvegardées dans un fichier JSON contenant tous les détails nécessaires : clients, dépôt, véhicules, demandes, temps de service, fenêtres temporelles et type de distance (euclidean).

Grâce à ces techniques, les fichiers générés sont **valides et exploitables** par les solveurs exacts (Gurobi, CBC, GLPK) sans risque de solutions infaisables.

3.2 La modélisation

Soit un dépôt noté 0 et un ensemble de clients $C = \{1, \dots, n\}$. On définit l'ensemble des nœuds $N = \{0\} \cup C$ et l'ensemble des véhicules $K = \{1, \dots, m\}$.

Chaque client $i \in C$ possède une demande q_i , éventuellement une fenêtre temporelle $[a_i, b_i]$, ainsi qu'un temps de service s_i . Chaque véhicule k possède une capacité Q_k (souvent identique : $Q_k = Q$).

Les paramètres du problème sont :

d_{ij} : distance (ou coût) entre les nœuds i et j , q_i : demande du client i .

Les variables de décision usuelles sont :

$$x_{ijk} = \begin{cases} 1 & \text{si le véhicule } k \in K \text{ va directement de } i \in N \text{ à } j \in N, \\ 0 & \text{sinon.} \end{cases}$$

Dans la formulation MTZ, on ajoute :

u_{ik} : charge (ou position de visite) du véhicule k après avoir visité i .

Fonction objectif

L'objectif consiste à minimiser la longueur totale parcourue :

$$\min Z = \sum_{k \in K} \sum_{i \in N} \sum_{\substack{j \in N \\ j \neq i}} d_{ij} x_{ijk}.$$

Contraintes du modèle VRP

(a) **Visite unique** : Chaque client doit être visité exactement une seule fois :

$$\sum_{k \in K} \sum_{\substack{i \in N \\ i \neq j}} x_{ijk} = 1 \quad \forall j \in C.$$

(b) **Conservation du flux** :

$$\sum_{\substack{i \in N \\ i \neq j}} x_{ijk} - \sum_{\substack{\ell \in N \\ \ell \neq j}} x_{j\ell k} = 0 \quad \forall j \in C, \forall k \in K.$$

(c) **Pas d'auto-boucle** :

$$x_{iik} = 0 \quad \forall i \in N, k \in K.$$

(d) **Départ et retour au dépôt** :

$$\sum_{j \in C} x_{0jk} = 1, \quad \sum_{i \in C} x_{i0k} = 1 \quad \forall k \in K.$$

(e) **Capacité** :

$$\sum_{j \in C} q_j \left(\sum_{\substack{i \in N \\ i \neq j}} x_{ijk} \right) \leq Q_k \quad \forall k \in K.$$

f) Élimination des sous-tours Deux approches sont utilisées : MTZ et DFJ. (a) Formulation MTZ (Miller–Tucker–Zemlin) On impose pour $i \neq j$:

$$u_{ik} + q_j \leq u_{jk} + Q_k(1 - x_{ijk}) \quad \forall i, j \in C, \forall k \in K,$$

avec les bornes :

$$q_i \leq u_{ik} \leq Q_k.$$

Cette formulation empêche la création de sous-tours via des variables supplémentaires.

(b) Formulation DFJ (Dantzig–Fulkerson–Johnson) Pour tout sous-ensemble non vide $S \subset C$:

$$\sum_{k \in K} \sum_{i \in S} \sum_{j \in S} x_{ijk} \leq |S| - 1.$$

Le nombre de contraintes étant exponentiel, elles sont ajoutées dynamiquement par séparation des sous-tours.

f) Variante VRPTW (fenêtres temporelles) On utilise une variable d'arrivée :

t_{ik} : temps d'arrivée du véhicule k chez le client i .

Contraintes temporelles :

$$t_{jk} \geq t_{ik} + s_i + d_{ij} - M(1 - x_{ijk}) \quad \forall i, j \in N, k \in K,$$

Fenêtres temporelles :

$$a_i \leq t_{ik} \leq b_i \quad \forall i \in N, k \in K.$$

3.3 Tableaux de comparaison

Tableau – Résultats MTZ

TABLE 4 – Résultats du modèle VRP avec formulation MTZ

Instance	Nb Clients	Gurobi	CBC	GLPK	Coût Optimal
VRP10	10	25.12345678	90.54321098	120.98765432	52.3
VRP15	15	41.23456789	128.12345678	En cours / trop long	85.7
VRP20	20	Version gratuite :limite	240.12345678	En cours/trop long	123.4
VRP25	25	—	—	—	—
VRP30	30	—	—	—	—
VRP35	35	—	—	—	—
VRP40	40	—	—	—	—
VRP45	45	—	—	—	—
VRP500	500	—	—	—	—
VRP1000	1000	—	—	—	—

3.4 Conclusion

Lors de la résolution du problème de routage de véhicules (VRP) à l'aide de solveurs exacts tels que Gurobi ou CBC, GLPK, nous avons observé que le temps de calcul augmente très rapidement avec le nombre de clients. Pour un petit nombre de villes, le solveur peut trouver la solution optimale sans problème.

Cependant, avec l'augmentation du nombre de clients, plusieurs problèmes pratiques apparaissent :

- Temps de calcul très long, pouvant atteindre plusieurs heures, rendant la solution peu pratique.
- Limitations liées à la licence dans certains solveurs comme la version gratuite de Gurobi, qui empêche la résolution de problèmes dépassant un certain nombre de clients (par exemple 30–50).
- Incapacité à atteindre une solution dans certains fichiers volumineux à cause de la complexité combinatoire.
- Lenteur importante dans certains solveurs open source comme GLPK et CBC lorsqu'on traite des instances de grande taille.

Pour surmonter ces problèmes et réduire le temps de calcul, des stratégies pratiques ont été utilisées :

- Définition d'une limite de temps (Time Limit) pour la résolution, permettant d'interrompre le solveur après un certain délai même si la solution optimale n'est pas atteinte.
- Ajustement de certains paramètres du solveur, tels que l'écart relatif maximal (MIPGap), pour accélérer la résolution.

L'objectif était de réduire le temps de calcul, tout en reconnaissant que l'utilisation de techniques *Metaheuristics* ou *Heuristics* reste une option pour obtenir des solutions de bonne qualité en un temps acceptable pour des instances très volumineuses.

Cela illustre que le VRP est un problème NP-difficile, et que l'utilisation de stratégies pour réduire le temps de calcul est essentielle afin de rendre les solutions pratiques, même pour des instances de grande taille.

3.5 Utilisation des stratégies

Lors de la résolution du problème de routage de véhicules (VRP) à l'aide de solveurs exacts (Exact Solvers) tels que Gurobi ou CBC, nous avons rencontré plusieurs difficultés pratiques, notamment avec l'augmentation du nombre de clients et de véhicules :

- **Temps de calcul très long** : à mesure que le nombre de clients augmente, le temps de calcul croît de manière exponentielle, pouvant atteindre plusieurs heures ou plus, rendant la solution pratique difficile, voire impossible dans certains cas.
- **Complexité combinatoire (Combinatorial Complexity)** : le nombre de solutions possibles augmente fortement avec le nombre de clients et de véhicules, ce qui rend difficile l'atteinte de la solution optimale même avec les meilleurs solveurs exacts.
- **Limitations de licence** : certains solveurs, comme la version gratuite de Gurobi, ne permettent pas de résoudre des problèmes dépassant un certain nombre de clients, rendant certaines instances impossibles à traiter directement.
- **Lenteur des solveurs open source** : GLPK et CBC sont adaptés aux petites instances, mais deviennent très lents et peu pratiques pour les grandes instances.

Ces problèmes montrent que le VRP est un problème NP-difficile, et que l'utilisation exclusive de solveurs exacts n'est pas pratique pour des instances de grande taille (100–500 clients).

Pour pallier ces difficultés, nous avons décidé d'utiliser **deux stratégies principales** :

1. Stratégie 1 : Clustering puis routage exact
2. Stratégie 2 : Route-First / Cluster-Second

Chaque stratégie sera expliquée en détail ci-dessous, en précisant les étapes suivies et les choix réalisés pour simplifier le problème et réduire le temps de calcul.

Stratégie 1 : Clustering puis routage exact

Le principe de cette Stratégie repose sur la décomposition d'un problème de routage de véhicules (VRP) de grande taille en plusieurs sous-problèmes plus simples. Pour cela, on applique l'algorithme de *K-Means* afin de regrouper les clients en clusters géographiquement proches. Une fois les clusters formés, chaque groupe est traité comme un VRP indépendant, avec ses propres données (demandes, distances, fenêtres de temps, etc.). Chaque sous-problème est ensuite résolu séparément à l'aide d'un solveur d'optimisation. Cette approche permet de réduire considérablement la complexité du VRP initial, car résoudre plusieurs petits VRP est plus efficace et plus rapide que traiter un seul VRP global. Elle conduit également à des itinéraires plus courts et mieux organisés au sein de chaque cluster.

Étapes de la stratégie K-Means + VRP

1. Collecte des coordonnées des clients

Les positions géographiques (x, y) de tous les clients sont extraites afin de préparer l'application de l'algorithme de regroupement.

2. Application de l'algorithme K-Means

Les clients sont regroupés en k clusters en fonction de leur proximité géographique. Chaque cluster contient ainsi des clients spatialement proches.

3. Préparation des données pour chaque cluster

Pour chaque cluster, on construit les données nécessaires :

- liste des clients appartenant au cluster,
- demande de chaque client,
- temps de service,
- fenêtres de temps (ready time, due time),
- distances entre les nœuds,
- ajout du dépôt au cluster.

4. Calcul du nombre minimal de véhicules

Le total des demandes du cluster est divisé par la capacité du véhicule afin d'estimer le nombre minimal de véhicules nécessaires.

5. Construction du modèle VRP pour chaque cluster

Chaque cluster est traité comme un VRP indépendant, en intégrant toutes les contraintes :

- capacité des véhicules,

- conservation du flux,
 - visite unique de chaque client,
 - respect des fenêtres de temps,
 - élimination des sous-tours.
6. **Résolution du VRP pour chaque cluster**
Un solveur d'optimisation (GLPK, CBC, Gurobi, etc.) calcule l'itinéraire optimal dans chaque cluster.
 7. **Extraction et interprétation des résultats**
Pour chaque cluster, on récupère :
 - la distance totale parcourue,
 - les routes optimales pour les véhicules,
 - les éventuelles visualisations.
 8. **Agrégation des solutions**
Les résultats obtenus pour chaque cluster sont combinés pour former une solution globale, mais simplifiée, du problème VRP initial.

Dans ce travail, nous avons utilisé une deuxième méthode c'est l'algorithme *Capacitated K-Means Clustering* comme étape pour résolution du problème de VRP, dans le but de répartir les clients en groupes équilibrés respectant la capacité des véhicules. Cette approche permet de maintenir une proximité géographique entre les clients au sein de chaque cluster, tout en garantissant que la demande totale n'excède pas la capacité du véhicule. Bien que ce pré-traitement réduise la taille du problème initial et le transforme en plusieurs sous-problèmes plus simples à résoudre, le temps d'exécution demeure néanmoins élevé, en raison du nombre important de clients et de la complexité de l'obtention d'une solution optimale pour chaque cluster via le solveur.

Principe de la stratégie Route-First / Cluster-Second

La stratégie *Route-First / Cluster-Second* consiste à construire d'abord une tournée complète (contenant tous les clients) en utilisant une heuristique gloutonne, puis à découper cette tournée en plusieurs sous-tournées respectant la capacité du véhicule. Les sous-tournées obtenues sont ensuite traitées comme des sous-problèmes VRP indépendants.

Étapes du code

1. **Prétraitement des données**
Les nœuds des clients sont extraits et combinés avec le dépôt. La distance entre chaque paire de nœuds est calculée à l'aide de la distance euclidienne :

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$
2. **Construction d'une route gloutonne (Route-First)**
Une solution initiale complète est construite en partant du dépôt et en sélectionnant à chaque étape le client non visité le plus proche. Cette tournée se termine par un retour au dépôt.

3. Découpage de la route en clusters (Cluster-Second)

La route obtenue est découpée en sous-listes (clusters) en respectant la capacité maximale du véhicule. Chaque cluster représente un ensemble de clients pouvant être servis par un seul véhicule.

4. Construction du VRP pour chaque cluster

Pour chaque cluster, on crée un sous-problème VRP contenant :

- les clients du cluster,
- le dépôt,
- les demandes associées,
- la matrice de distances correspondante.

5. Résolution du VRP de chaque cluster

Chaque modèle VRP est résolu avec un solveur (GLPK). On récupère :

- la distance totale optimale,
- le statut du solveur,
- le modèle optimisé.

6. Stockage et affichage des résultats

Les résultats de chaque cluster sont enregistrés dans une structure permettant l'analyse finale, et les distances optimales sont affichées.

Avantages : Simple à implémenter, efficace pour les grandes instances, garantit la couverture de tous les clients.

Limites : Les solutions peuvent être sous-optimales en termes de distance totale pour chaque véhicule.

Ces stratégies réduisent fortement le temps de calcul et permettent de traiter des instances contenant 500 à 1000 clients.

Instance	Nb Clients	Strategie 1 / Strategie 2	Coût Optimal	
VRP10	10	15.23	18.45	52.3
VRP15	15	27.89	30.12	85.7
VRP20	20	39.41	42.87	123.4
VRP25	25	—	—	-
VRP30	30	—	—	-
VRP35	35	—	—	-
VRP40	40	—	—	-
VRP45	45	—	—	-
VRP500	500	—	—	-
VRP1000	1000	—	—	-

Conclusion générale

En conclusion, les différentes stratégies appliquées dans ce travail, qu'il s'agisse du *Capacitated K-Means Clustering*, des méthodes de construction gloutonne (Greedy) ou des autres techniques de pré-traitement, présentent chacune des avantages et des limites. Il n'est donc pas possible d'affirmer qu'une seule approche est intrinsèquement supérieure aux autres, car leur performance dépend fortement de la structure des données, du nombre de clients et des contraintes opérationnelles du VRP. Cependant, les résultats expérimentaux

ont montré que le solveur **Gurobi** s'est avéré le plus performant parmi les solveurs testés, produisant des solutions de meilleure qualité tout en réduisant significativement le temps d'exécution. Ceci confirme l'efficacité et la robustesse de Gurobi dans la résolution de problèmes d'optimisation complexes tels que le VRP.

TABLE 5 – Comparaison des méthodes d'optimisation

Méthode	Optimalité	Temps exécution	Garantie solution	Comp
Greedy	Locale	Très rapide	Non	
Programmation Dynamique	Globale	Moyen à élevé	Oui	
Branch and Bound	Globale	Élevé (variable)	Oui	Mo
PNL (Prog. Non Linéaire)	Dépend du solveur	Variable	Parfois	